Java Full Stack August 2026 Capstone Presentation

Team 1: Brandon Robare, Bryan Caskey,
Carter Bossong, Chad Wangolo

Overview

- User Stories
- NFRs
- Architecture Diagram
- Front End
- Messaging
- Persistence
- Deployment
- AI Usage
- Retrospective
- Future Goals

User Stories



EPIC: Agent adds an interaction for a user, it gets saved, and an event goes to Kafka.



User Stories:

As a user, I should be able to create an interaction.

As a user, I should be able to log in.

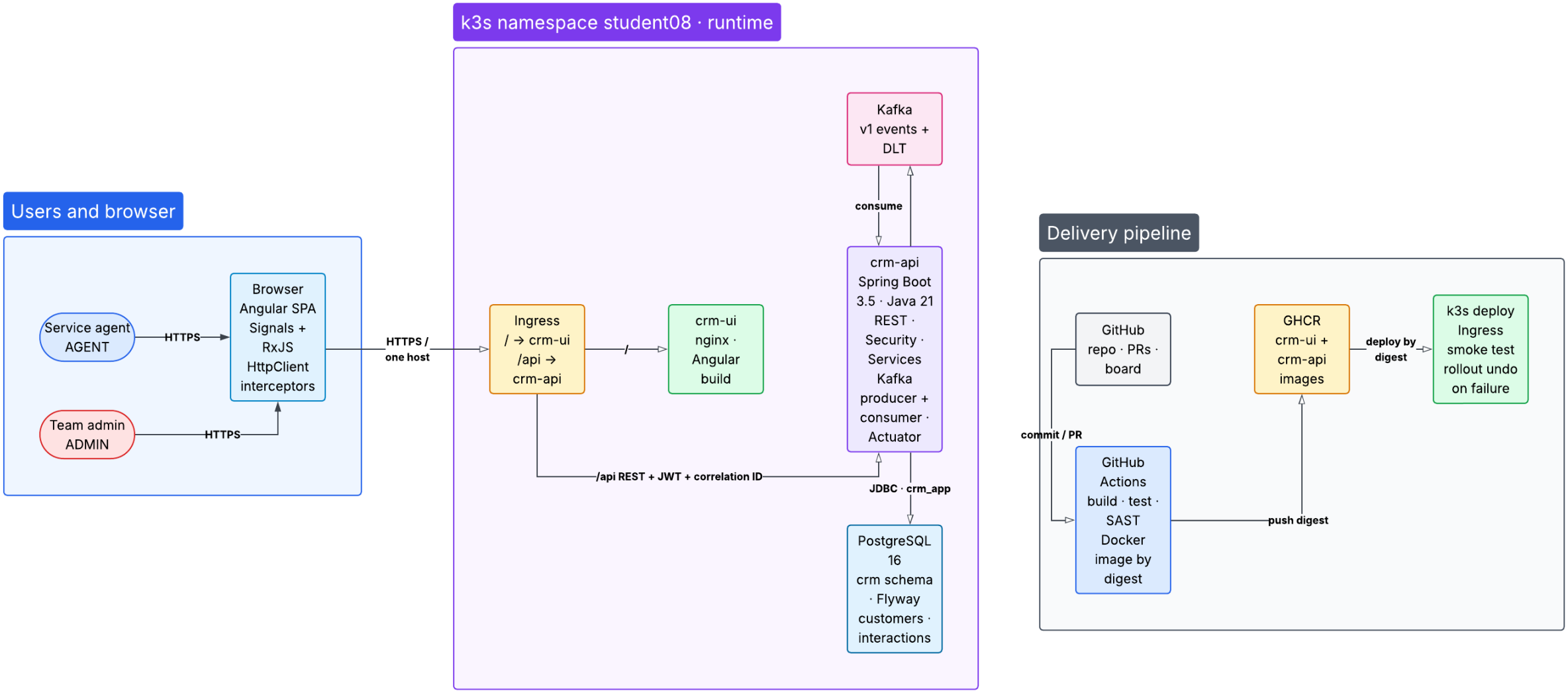
As an agent, I should be able to view interactions.

As an admin, I should be able to view a list of customers.

NFRs

- Security & Authorization: Secure API boundary using 30-minute JWTs; missing tokens return 401, wrong roles return 403.
- Data Integrity & Durability: Commits are saved permanently in PostgreSQL even if the API; no sensitive summaries or PII are published to Kafka or logged.
- Performance Targets: Key UI workflows (search, profiles, saving interactions) load under 2 second in 95% of cases; search powered by database index scans.
- Accessibility & Compatibility: Designed with semantic HTML and plain CSS for keyboard navigation, screen reader support, and responsive layouts across
- Observability & CI/CD: End-to-end request tracking using X-Correlation-ID across logs and Kafka events; zero hardcoded secrets with single-digest Docker deployments

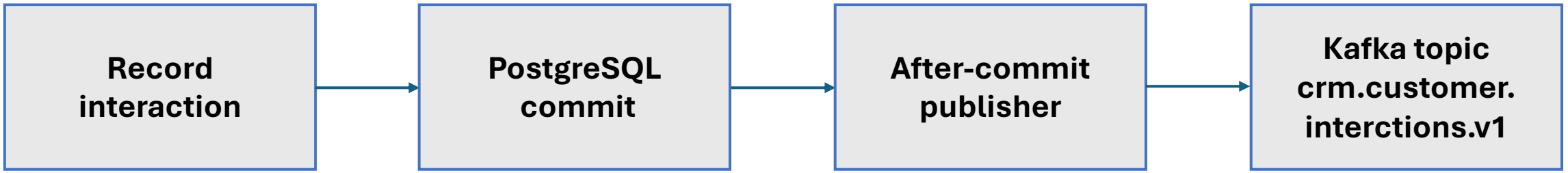
Architecture Diagram



Front End

- Angular SPA
- JWT Bearer Authentication
- RxJS
- NGINX

Messaging

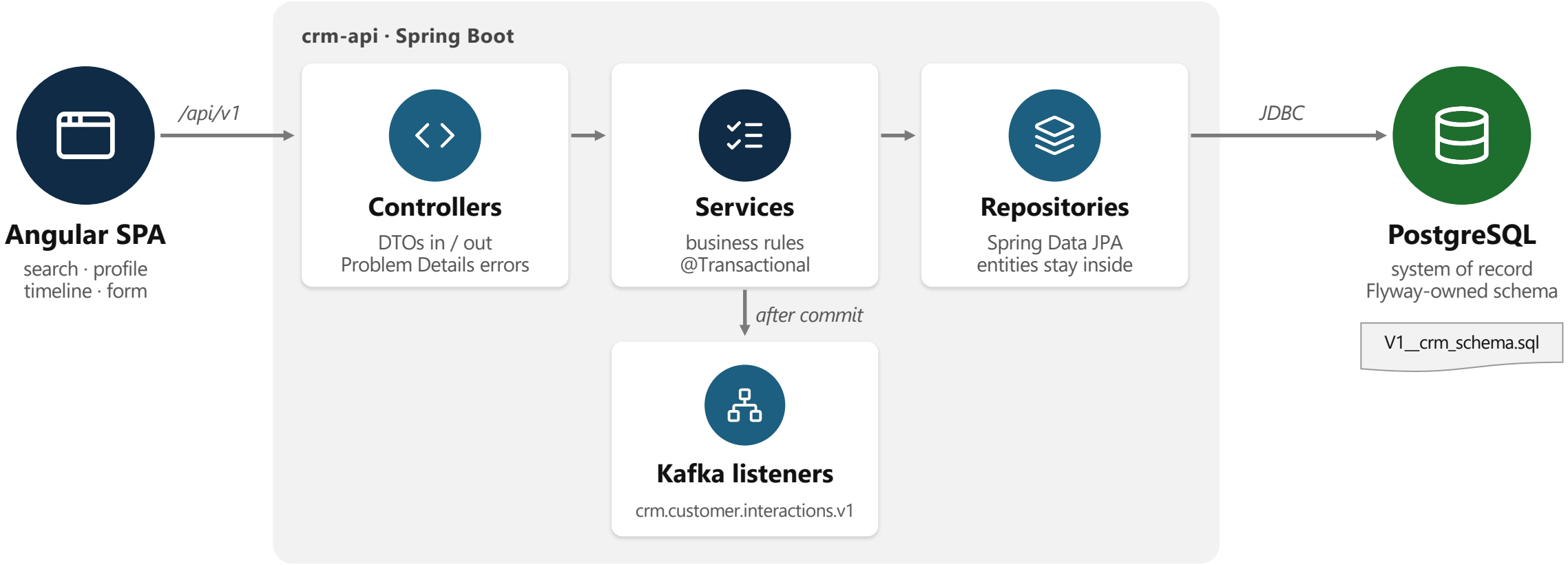


- Publishes only after a successful database commit

- Correlation ID links requests -> logs -> interaction event

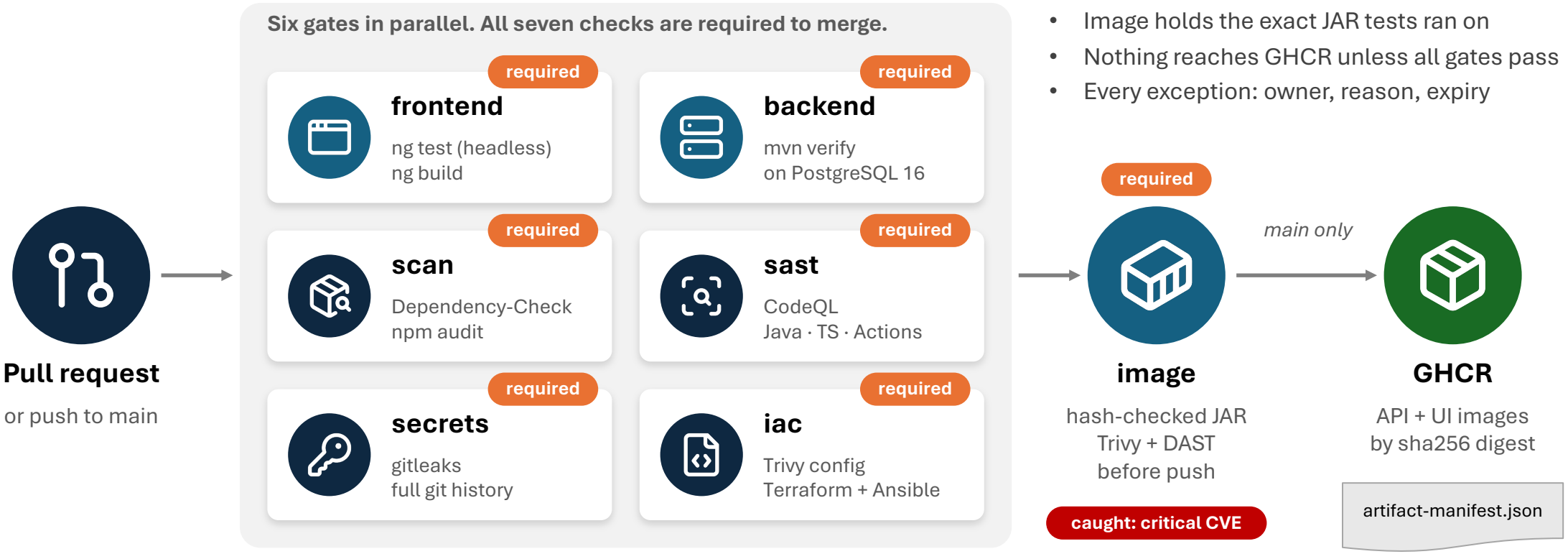
Persistence

One request path, four layers. Business rules in one.



Continuous Integration

One path for every change. Nothing is published unless every gate passes.



- Image holds the exact JAR tests ran on
- Nothing reaches GHCR unless all gates pass
- Every exception: owner, reason, expiry

Continuous Delivery

A person decides when it ships. Production runs the exact digest CI built.



Green main run
API + UI digests

Tag v0.1.4
by the release owner

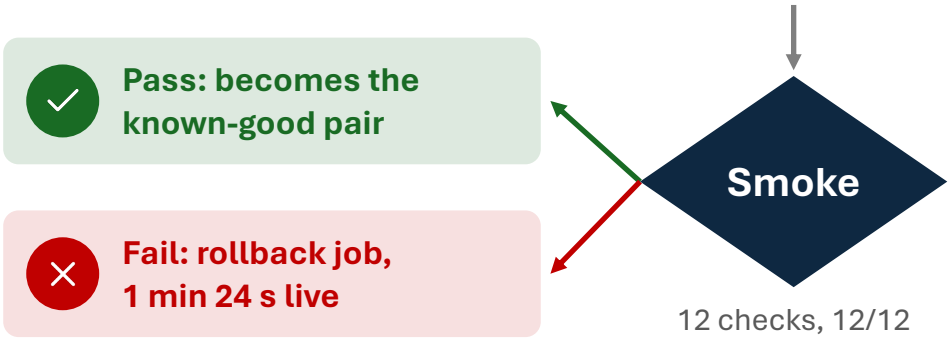
Approval
recorded in GitHub

Provenance check
same digests, no drift

Deploy by digest
backup first, API then UI

CD refuses to ship

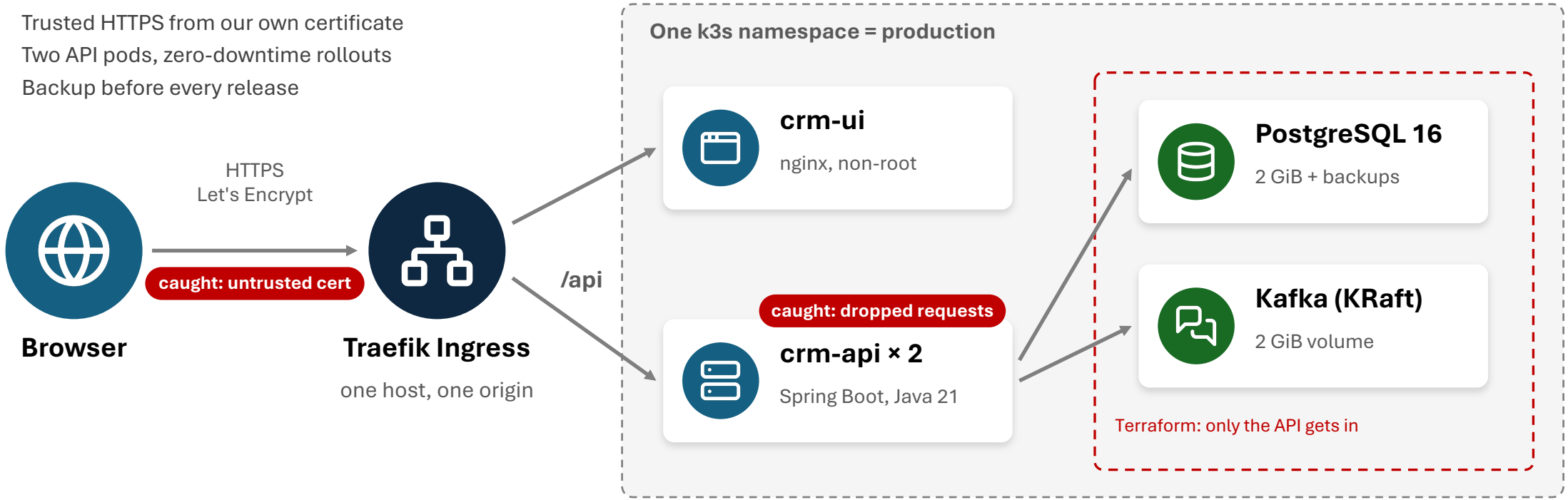
- ":latest" or any tag
- another commit's build
- an image not from GHCR
- skipping smoke
- pending Terraform drift — caught on v0.1.4
- a failed backup



Production on k3s

Beyond the brief

- Trusted HTTPS from our own certificate
- Two API pods, zero-downtime rollouts
- Backup before every release





Backup: security gates

Gate	Tool	Fails at	Stops	Exception
Java dependencies	OWASP Dependency-Check 12.2.2	CVSS 7+	merge + publish	scoped suppression
npm, runtime deps	npm audit --omit=dev	high	merge + publish	GHSA ID with expiry
Static analysis	CodeQL (Java, TS, Actions)	severity 7.0+	merge + publish	rule + file with expiry
Secrets	gitleaks 8.30.1, full history	any finding	merge + publish	reasoned ignore entry
Kubernetes config	Trivy config	high	publish	trivyignore entry
Container image	Trivy 0.75.0, before push	critical	publish	version-scoped entry

Every exception has an owner, a reason and an expiry in docs/security-findings.csv. Thresholds are never lowered to go green.

Evidence index: Dependency scan + triage · Dependency gate · SAST gate · Secrets scan · Image gate (Trivy)

AI Usage

- **Allowed**
- Boilerplate code generation (DTOs, Flyway, Kafka, Angular)
- Test drafts
- HCL/YAML sketches
- Docs

- **Forbidden**
- Committing secrets or kubeconfig
- Applying Terraform/Ansible to prod without approval
- Copying proprietary third-party code

DEMO

Retrospective: Challenges

Changed to using k3s instead of OpenShift

Staging development was scrapped due to constraints

Limited time

Retrospective: What we would do different

Full CRUD operations on Customers and Interactions.

OpenShift deployment with staging area.

Blue-Green deployment method.

Thank You